

# Лекция 5. Bash-автоматизация и анализ системных данных

## Цель лекции

Понять структуру надежного административного Bash-сценария

Использовать переменные, аргументы, условия, циклы и функции для проверок Linux-сервера

Возвращать корректные exit codes для мониторинга и автоматического запуска

Обрабатывать ошибки, логировать события и безопасно очищать временные ресурсы

Собрать законченный сценарий `server-health.sh` для проверки диска и `systemd`-службы

## Учебные вопросы

1. Структура административного сценария, переменные, quoting и command substitution
2. Аргументы и \$?, \$#, \$@
3. if, case, логика, for, while и функции
4. Exit codes и мониторинг
5. set -euo pipefail, обработка ошибок и trap
6. Логирование; grep, sed, awk, sort, uniq
7. Идемпотентность и безопасный повторный запуск

# 1. Административный сценарий должен быть предсказуемым

Bash-сценарий автоматизирует повторяемые команды администратора Linux

Эксплуатационный сценарий должен быть читаемым, проверяемым и безопасным при повторном запуске

В начале задают interpreter line, окружение, константы и функции

Основная логика обычно вызывается через функцию main

В конце сценарий возвращает понятный exit code

В лаборатории собирается /usr/local/sbin/server-health.sh

**server-health.sh** только проверяет состояние сервера

Сценарий проверяет заполнение файловой системы и состояние systemd-службы

Он ничего не удаляет, не перезапускает и не исправляет автоматически

Read-only проверки безопаснее для cron, Zabbix и учебного Break/Fix-сценария

Автоматическое исправление лучше выносить в отдельный server-remediate.sh

Health Check сообщает состояние, а решение об исправлении принимает администратор или утвержденная политика

## Каркас сценария задает единый стиль

`#!/usr/bin/env bash` - запускать сценарий через Bash из окружения

`set -euo pipefail` - включить строгий режим с оговорками, которые будут разобраны отдельно

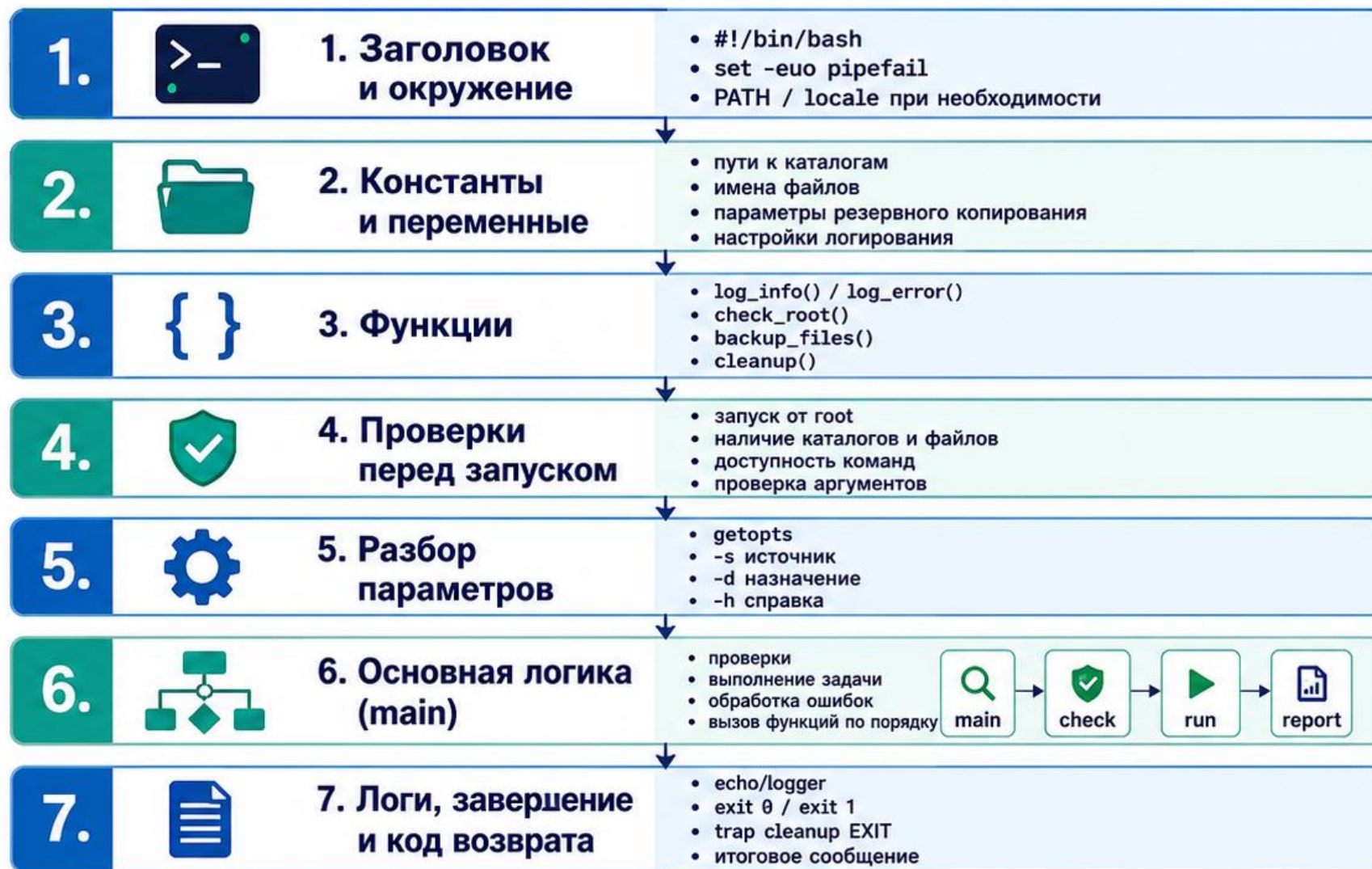
`readonly EXIT_OK=0` - константа успешного состояния

`main() { ... }` - основная функция сценария

`main "$@"` - передать функции исходные аргументы командной строки

Такой каркас легче тестировать, читать и расширять

# Структура административного Bash-сценария



## Принципы хорошего сценария



понятные имена



проверка ошибок



логирование действий



функции вместо длинного кода



предсказуемый код возврата



Общий шаблон



shebang



переменные



функции



проверки



разбор аргументов



main



exit



## Переменные заменяют магические числа и случайные строки

`SERVICE='nginx'` задает проверяемую службу по умолчанию

`MOUNTPOINT='/'` задает проверяемую точку монтирования

`DISK_WARN=80` задает порог предупреждения по диску

`DISK_CRIT=90` задает критический порог

`readonly EXIT_CRITICAL=2` делает код состояния понятным по имени

Пороговые значения лучше вынести в начало сценария или параметры запуска



## Quoting защищает сценарий от пробелов и подстановок shell

Quoting - заключение переменных в кавычки при подстановке

Плохо: `cd $path` - путь с пробелом разобьется на несколько слов

Лучше: `cd "$path"` - значение переменной передается как один аргумент

Проверка файла: `[[ -f "$file" ]]`

Незакавыченная переменная может попасть под word splitting и filename expansion

Практическое правило: почти всегда заключать переменные в двойные кавычки

## Одинарные и двойные кавычки работают по-разному

В двойных кавычках Bash подставляет переменные и command substitution

Пример: `printf 'service=%s\n' "$SERVICE"` сохраняет значение как один аргумент

В одинарных кавычках переменные не раскрываются

Пример: `echo '$SERVICE'` выведет буквальный текст `$SERVICE`

Одинарные кавычки удобны для awk-программ и простых trap-обработчиков

Выбор кавычек зависит от того, нужна ли подстановка переменных сейчас или позже

## **Command substitution подставляет вывод команды в переменную**

Command Substitution - подстановка вывода команды в другое выражение

Современная форма: `value=$(command)`

Пример: `host=$(hostname -s)`

Command substitution удаляет завершающие переводы строки из вывода

Вывод может содержать несколько строк, поэтому нельзя всегда считать его простым числом

Для сложных структур лучше читать строки через `while`  
`IFS= read -r` или использовать формат для машинной обработки

## Разделяйте объявление `local` и `command substitution`

Внутри функции переменные лучше объявлять через `local`

Плохо: `local used=$(df -P / | awk 'NR==2 {print $5}')`

Такой стиль может скрыть `exit code` команды за успешным выполнением `local`

Лучше: `local used`, затем `used=$(...)` отдельной строкой

После `command substitution` значение нужно проверить на ожидаемый формат

Это особенно важно для числовых сравнений и мониторинговых `exit codes`

## 2. Аргументы делают сценарий гибким

\$0 содержит имя запущенного сценария

\$1, \$2 и дальше содержат позиционные аргументы

\$# показывает количество аргументов

\$? показывает код завершения предыдущей команды

\$@ передает все исходные аргументы дальше

Для server-health.sh нужны ключи -h, -s SERVICE, -w WARNING и -c CRITICAL

## **"\$@" и "\$\*" передают аргументы по-разному**

"\$@" передает каждый исходный аргумент отдельно

Пример: `./script.sh "web server" 80` содержит два аргумента

`main "$@"` передаст в `main` два аргумента: `web server` и `80`

"\$\*" объединяет все аргументы в одну строку

Для передачи исходных аргументов другой функции почти всегда используют "\$@"

Незакавыченные `$@` и `$*` могут подвергаться `word splitting` и `filename expansion`

## **\$? нужно сохранять сразу после команды**

\$? равен 0, если предыдущая команда завершилась успешно

Ненулевое значение означает ошибку или особое состояние, заданное программой

Пример демонстрации: `systemctl is-active --quiet nginx;`  
`echo $?`

Любая следующая команда перезапишет \$?

Если код нужен, сохранить его нужно сразу:

`some_command; rc=$?`

При `set -e` ожидаемо неуспешные команды лучше выполнять внутри `if`



## В рабочем сценарии лучше проверять команду напрямую

Демонстрация через `echo $?` полезна для обучения

В рабочем коде безопаснее писать: `if systemctl is-active --quiet "$service"; then ...`

Так не нужно ловить `$?` отдельной строкой

Ожидаемая неактивная служба не завершит сценарий при `set -e`, если команда находится в `if`

Если код нужен, его можно сохранить в ветке `else`:  
`rc=$?`

Проверка команды должна соответствовать смыслу результата, а не только числу

## **getopts разбирает короткие параметры командной строки**

getopts - встроенный механизм Bash для обработки коротких ключей

Ключ -h показывает справку

Ключ -s nginx выбирает проверяемую systemd-службу

Ключ -w 80 задает порог WARNING

Ключ -c 90 задает порог CRITICAL

Ошибочные параметры должны завершаться понятным сообщением и кодом UNKNOWN

## **usage()** показывает правила запуска сценария

Функция `usage` печатает краткую справку для пользователя

Пример строки: Использование: `$0 [-h] [-s SERVICE] [-w WARN] [-c CRIT]`

`-h` - показать справку

`-s SERVICE` - проверяемая `systemd`-служба

`-w WARN` - порог WARNING в процентах

`-c CRIT` - порог CRITICAL в процентах

## Рабочий getopt должен обрабатывать ошибки параметров

Пример цикла: while getopt ':hs:w:c:' opt; do case "\$opt" in ... esac; done

h) usage; exit "\$EXIT\_OK" ;; - показать справку

s) SERVICE=\$OPTARG ;; - сохранить имя службы

w) DISK\_WARN=\$OPTARG ;; - сохранить WARNING-порог

c) DISK\_CRIT=\$OPTARG ;; - сохранить CRITICAL-порог

:) echo "UNKNOWN: параметр -\$OPTARG требует значение" && exit "\$EXIT\_UNKNOWN" ;;

## Числовые параметры нужно валидировать до проверок

Порог диска должен быть числом от 1 до 100

DISK\_WARN должен быть меньше DISK\_CRIT

Проверка формата: `[[ $value =~ ^[0-9]+$ ]]`

Проверка диапазона: `(( value >= 1 && value <= 100 ))`

Если параметр неверный, сценарий возвращает UNKNOWN, а не CRITICAL

Ошибка параметра означает проблему выполнения проверки, а не состояние сервера

### 3. if должен проверять смысл результата, а не только строку

if используется для проверки результата команды, числа, строки или файла

Пример команды: `if systemctl is-active --quiet "$service"; then ...`

Для чисел в `[[ ]]` удобнее использовать арифметику: `(( used >= crit ))`

Для строк применяют `[[ -n "$value" ]]` или `[[ "$state" == OK ]]`

Условие должно быть читаемым и соответствовать диагностическому смыслу

Неуспешный результат ожидаемой проверки не всегда

## case удобен для параметров и состояний

case заменяет длинную цепочку if при выборе по одному значению

В getopt case обрабатывает h, s, w, c и ошибки параметров

В сценарии мониторинга case может обрабатывать состояния OK, WARNING, CRITICAL и UNKNOWN

В конце case нужно предусмотреть неизвестное значение

Неизвестное состояние лучше завершать как UNKNOWN

Такой подход делает сценарий понятнее для

## **for подходит для заранее известного списка**

for удобен, когда список элементов известен заранее

Имена systemd units зависят от дистрибутива и пакета

В Debian/Ubuntu служба SSH часто называется ssh

В RHEL-подобных системах служба часто называется  
sshd

OpenVPN unit может называться openvpn-  
server@server

Перед лабораторией реальные имена проверяют:

```
systemctl list-unit-files | grep -E 'nginx|ssh|openvpn'
```



## Массивы безопаснее строки со списком служб

Bash array хранит элементы как отдельные значения

Пример: `SERVICES=(nginx ssh openvpn-server@server)`

Цикл: `for svc in "${SERVICES[@]}; do check_service  
"$svc"; done`

Кавычки вокруг `"${SERVICES[@]}"` сохраняют каждый элемент отдельно

Массивы уменьшают риск ошибки при пробелах и специальных символах

Для базового `server-health.sh` достаточно одной службы через `-s SERVICE`

## **while IFS= read -r сохраняет строку как есть**

while подходит для чтения строк файла или потока

Надежный шаблон: `while IFS= read -r line; do printf '%s\n' "$line"; done < servers.txt`

IFS= сохраняет начальные и завершающие пробелы

`read -r` не обрабатывает обратные слешы как escape-последовательности

Для последней строки без перевода строки можно добавить условие `[[ -n $line ]]`

Для базовой лекции достаточно освоить `IFS= read -r`

**Функция должна делать одну проверку и возвращать статус**

Функция объединяет одну логическую операцию  
check\_disk проверяет заполнение точки монтирования  
check\_service проверяет состояние systemd-службы  
printf или echo формирует сообщение для человека  
или мониторинга  
return возвращает статус функции вызывающему коду  
exit завершает весь сценарий и применяется в main  
или при фатальной ошибке

## **check\_service** разделяет сообщение и код возврата

Начало функции: `check_service() { local service=$1; ... }`

Проверка: `if systemctl is-active --quiet "$service"; then`

Успех: `printf 'OK: service %s is active\n' "$service"; return 0`

Отказ: `printf 'CRITICAL: service %s is inactive\n' "$service"; return 2`

Сообщение объясняет состояние, а return передает код мониторингу

Функция не должна делать exit, если ее результат еще нужно агрегировать

**check\_disk** должен удалить знак процента перед сравнением

Команда `df -P / | awk 'NR==2 {print $5}'` возвращает значение вида 82%

Сравнение [ "\$used" -ge 90 ] с 82% завершится ошибкой

Нужно удалить знак процента: `gsub(/%/,"",$5)`

Пример: `used=$(df -P "$mountpoint" | awk 'NR==2 {gsub(/%/,"",$5); print $5}')`

После извлечения значение проверяют регулярным выражением `^[0-9]+$`

Если значение не число, функция возвращает UNKNOWN

**check\_disk** должен знать, какую файловую систему проверяет

Для начала лучше проверять конкретную точку монтирования, например /

Команда `df -P "$MOUNTPOINT"` дает POSIX-совместимый формат вывода

Если проверяются все ФС, нужно исключать `tmpfs`, `devtmpfs`, `squashfs` и контейнерные mounts

Сетевые файловые системы проверяются отдельно, если это входит в задачу

Иначе сценарий будет выдавать шумные или ложные предупреждения

Для лабораторного `server-health.sh` достаточно



# Схема: функции server-health.sh

Сценарий проверяет состояние сервера (диск и systemd-службу) и возвращает понятный exit code



## 4. Exit codes связывают Bash-сценарий с мониторингом

Exit Code - числовой код завершения процесса

Общее Unix-правило: 0 означает успех, ненулевой код означает неуспех или особое состояние

Конкретные ненулевые значения определяет сама программа

Допустимый exit code процесса находится в диапазоне 0-255

Мониторинговые системы читают и текст, и код завершения

Поэтому `server-health.sh` должен печатать понятное сообщение и завершаться правильным кодом



## **Коды 0-3 являются соглашением мониторингового сценария**

В лаборатории используется схема, совместимая с подходом Nagios/Icinga plugins

0 = OK: проверка прошла успешно

1 = WARNING: параметр близок к опасному значению

2 = CRITICAL: требуется действие администратора

3 = UNKNOWN: сценарий не смог корректно

выполнить проверку

Эта схема не является универсальным стандартом для всех Linux-команд

## Несколько проверок нужно агрегировать в один итог

Если диск WARNING, а служба CRITICAL, общий результат должен быть CRITICAL

Если команда проверки не смогла выполниться, возможен UNKNOWN

Приоритет должен быть определен явно до написания кода

Простое числовое сравнение делает UNKNOWN=3 выше CRITICAL=2

Это может быть допустимо, но должно быть осознанным решением

В учебном `server-health.sh` используем порядок

## Агрегация статусов выполняется после вызова функций

Функция `check_disk` возвращает `disk_rc`

Функция `check_service` возвращает `service_rc`

Если `disk_rc` или `service_rc` равен 2, сценарий завершится `EXIT_CRITICAL`

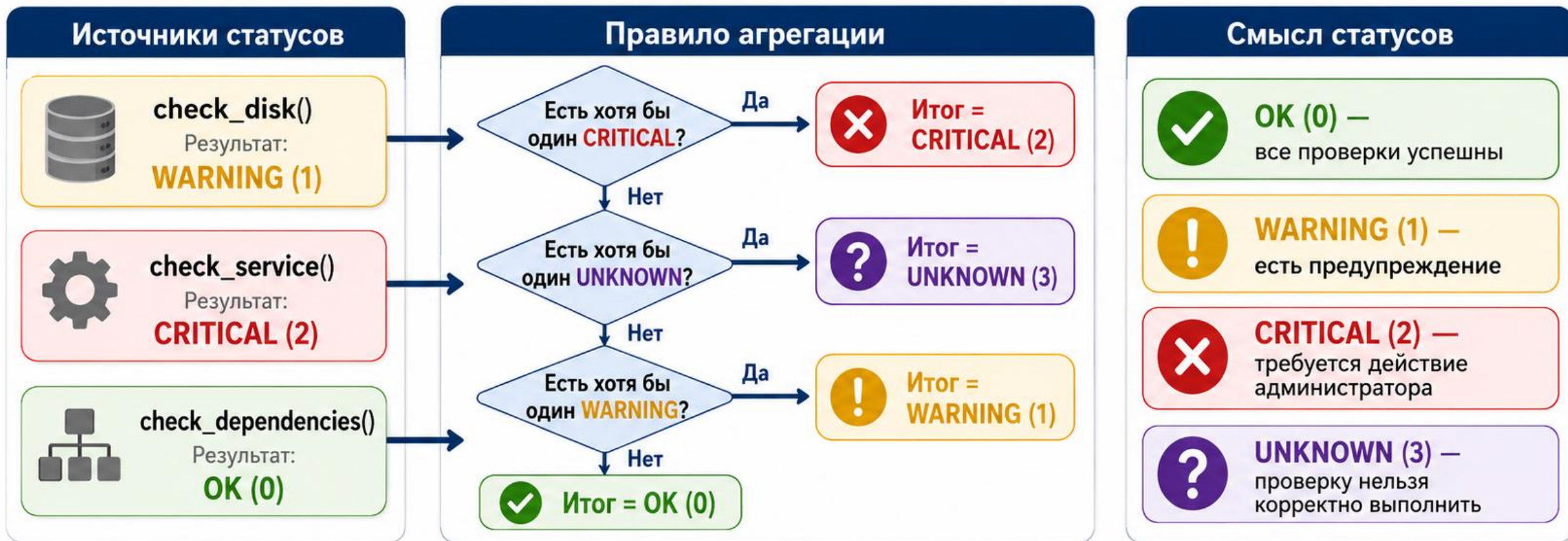
Если критичных ошибок нет, но есть `UNKNOWN`, сценарий завершится `EXIT_UNKNOWN`

Если есть `WARNING` и нет более серьезных состояний, сценарий завершится `EXIT_WARNING`

Если все проверки ОК, сценарий завершится `EXIT_OK`

# Диаграмма: агрегация OK, WARNING, CRITICAL и UNKNOWN

Как server-health.sh объединяет результаты нескольких проверок в один итоговый статус



**Ключевая идея**

Сценарий возвращает один итоговый exit code для мониторинга, cron, Zabbix, Nagios/Icinga.

## 5. set -е помогает, но не заменяет явную обработку ошибок

set -е помогает остановить сценарий при  
необработанной ошибке простой команды

Но у set -е есть исключения внутри if, while, until, &&  
|| и некоторых pipeline-контекстов

Ожидаемо неуспешная проверка службы должна быть  
внутри if

Опасно: `systemctl is-active --quiet nginx; status=$?` при  
set -е

Сценарий может завершиться до сохранения status

Правильно: `if systemctl is-active --quiet "$service"; then`  
`... else rc=$?; fi`



**set -u требует значений по умолчанию или явной проверки**

set -u завершает сценарий при обращении к необъявленной переменной

Значение по умолчанию: `threshold=${THRESHOLD:-80}`

Обязательная переменная: `: "${target_dir:?target_dir is not set}"`

Для разрушительных путей лучше аварийно завершиться, чем выбирать default

Пример опасности: `rm -rf "$target_dir"`, если `target_dir` не определена

Строгий режим полезен только вместе с осознанными проверками

## **pipefail меняет смысл pipeline-ошибок**

Без pipefail pipeline возвращает код последней команды

Пример: false | true завершится кодом 0 без pipefail

С set -o pipefail pipeline вернет ненулевой статус, если одна из команд упала

Практический пример: journalctl -u nginx | grep -q error

Нужно различать: журнал недоступен, grep не нашел совпадение, grep обнаружил ошибку

pipefail делает ошибки видимее, но не заменяет смысловую обработку состояний

## trap должен безопасно очищать временные файлы

trap - механизм запуска команды или функции при выходе или сигнале

Корректный простой пример: `tmpfile=$(mktemp); trap 'rm -f "$tmpfile"' EXIT`

Надежнее использовать функцию `cleanup`

```
cleanup() { rm -f -- "${tmpfile:-}"; }
```

Конструкция `${tmpfile:-}` не падает при `set -u`, если переменная еще не определена

`trap cleanup EXIT INT TERM` выполняет очистку при завершении и типовых сигналах



**mktemp** должен проверяться как обычная команда

mktemp создает уникальный временный файл или каталог

Пример файла: `tmpfile=$(mktemp) || { echo 'UNKNOWN: не удалось создать временный файл' >&2; exit 3; }`

Пример каталога: `tmpdir=$(mktemp -d)`

Перед `rm -rf` нужно убедиться, что путь не пустой и создан именно `mktemp`

Для базового `server-health.sh` временный файл не обязателен

Но умение писать `cleanup` пригодится для сценариев с

## **ERR-trap полезен для диагностики, но имеет ограничения**

Обработчик ошибок может печатать строку и exit code

Пример идеи: `trap 'on_error "$LINENO"' ERR`

ERR имеет похожие контекстные ограничения с `set -e`

Для наследования в функциях и subshell иногда используют `set -E`

Такой обработчик помогает найти место аварии

Но он не заменяет явные сообщения UNKNOWN в ожидаемых ошибках проверок

## 6. Логирование нужно разделять на STDOUT, STDERR и Journald

STDOUT - итоговое состояние для мониторинга: OK:

disk=42%, nginx=active

STDERR - ошибки выполнения: echo 'UNKNOWN: df command failed' >&2

Journald или Syslog - история запусков сценария

logger -t server-health -- "disk usage: \${used}%"

отправляет запись в системный журнал

Проверка журнала: journalctl -t server-health

В логах нельзя хранить пароли, токены, private keys и содержимое credential-файлов

## Файловый лог требует владельца, прав и ротации

Файл `/var/log/server-health.log` удобен, но требует сопровождения

Нужно определить владельца и права доступа

Нужно настроить `logrotate` или ограничение объема

Нужно учитывать риск `symlink attacks` при записи от `root`

Для учебной работы проще использовать `logger` и системный журнал

`Journald` сам добавляет временную метку и источник сообщения

## grep, sed и awk должны решать конкретные задачи

grep ищет строки по шаблону, например `grep -i error` в журнале

sed выполняет потоковое редактирование, например удаление лишнего префикса

awk разбирает строки по полям и выполняет простую обработку

Пример для диска: `df -P / | awk 'NR==2 {gsub(/%/,"", $5); print $5}'`

Перед включением в сценарий команду нужно проверить на реальном выводе

Если формат вывода нестабилен, сценарий должен

**sort и uniq помогают анализировать повторяющиеся значения**

sort сортирует строки и ставит одинаковые значения рядом

uniq удаляет или считает только соседние одинаковые строки

Перед uniq почти всегда нужен sort

Пример: `cut -d ':' -f7 /etc/passwd | sort | uniq -c`

Так можно увидеть, какие shell назначены пользователям

Для логов похожая схема помогает считать повторяющиеся IP, коды ответа или имена служб

## 7. Идемпотентность делает повторный запуск безопасным

Идемпотентность - свойство команды или сценария давать безопасный результат при повторном запуске

Команда `mkdir -p /var/log/server-health` уже идемпотентна для существующего каталога

Поэтому проверка `[ -d dir ]` перед `mkdir -p` обычно избыточна

Настоящая проблема - команды, которые каждый запуск добавляют строку повторно

Сценарий `cron` может запускаться много раз, поэтому повторный запуск должен быть штатным сценарием

Проверки состояния должны быть отделены от

## Идемпотентное добавление строки требует проверки содержимого

Учебный файл: `file='/tmp/app.conf'`

Учебная строка: `line='enabled=true'`

Проверка: `grep -qxF "$line" "$file" 2>/dev/null`

Добавление только при отсутствии: `printf '%s\n' "$line"`  
`>> "$file"`

Для `/etc/hosts` или системных конфигураций нужен  
backup и отдельное разрешение на изменение

Health Check обычно не должен менять конфигурацию  
сам



## Cron создает минимальное и непривычное окружение

Сценарий может работать вручную, но падать из cron

Причины: короткий PATH, другой рабочий каталог, отсутствие TTY и переменных окружения

В начале сценария полезно задать

```
PATH='/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin'
```

После этого выполнить `export PATH`

Не нужно полагаться на текущий каталог

Пути к файлам и логам лучше задавать абсолютными

**sudo** внутри автоматического сценария может зависнуть

server-health.sh в /usr/local/sbin обычно запускается  
cron, Zabbix или systemd timer

Если внутри сценария стоит sudo, он может запросить  
пароль и зависнуть

Лучше запускать сценарий от пользователя с нужными  
правами

Для read-only проверок не всегда нужен root

Если root обязателен, можно проверить EUID в начале  
сценария

Пример: if (( EUID != 0 )); then echo 'UNKNOWN: нужен  
root' >&2; exit 3; fi

## **flock защищает от параллельного запуска**

Идемпотентность не предотвращает одновременный запуск двух экземпляров

Cron может запустить новый экземпляр, пока предыдущий еще работает

Пример lock descriptor: `exec 9>/run/lock/server-health.lock`

Проверка: `if ! flock -n 9; then echo 'UNKNOWN: другой экземпляр уже работает' >&2; exit 3; fi`

Lock особенно важен при записи общего файла, отправке отчета или очистке

Для read-only коротких проверок lock может быть не

## Сценарий нужно проверять до запуска на сервере

`bash -n /usr/local/sbin/server-health.sh` проверяет синтаксис без выполнения

ShellCheck анализирует качество Bash-кода и типовые ошибки

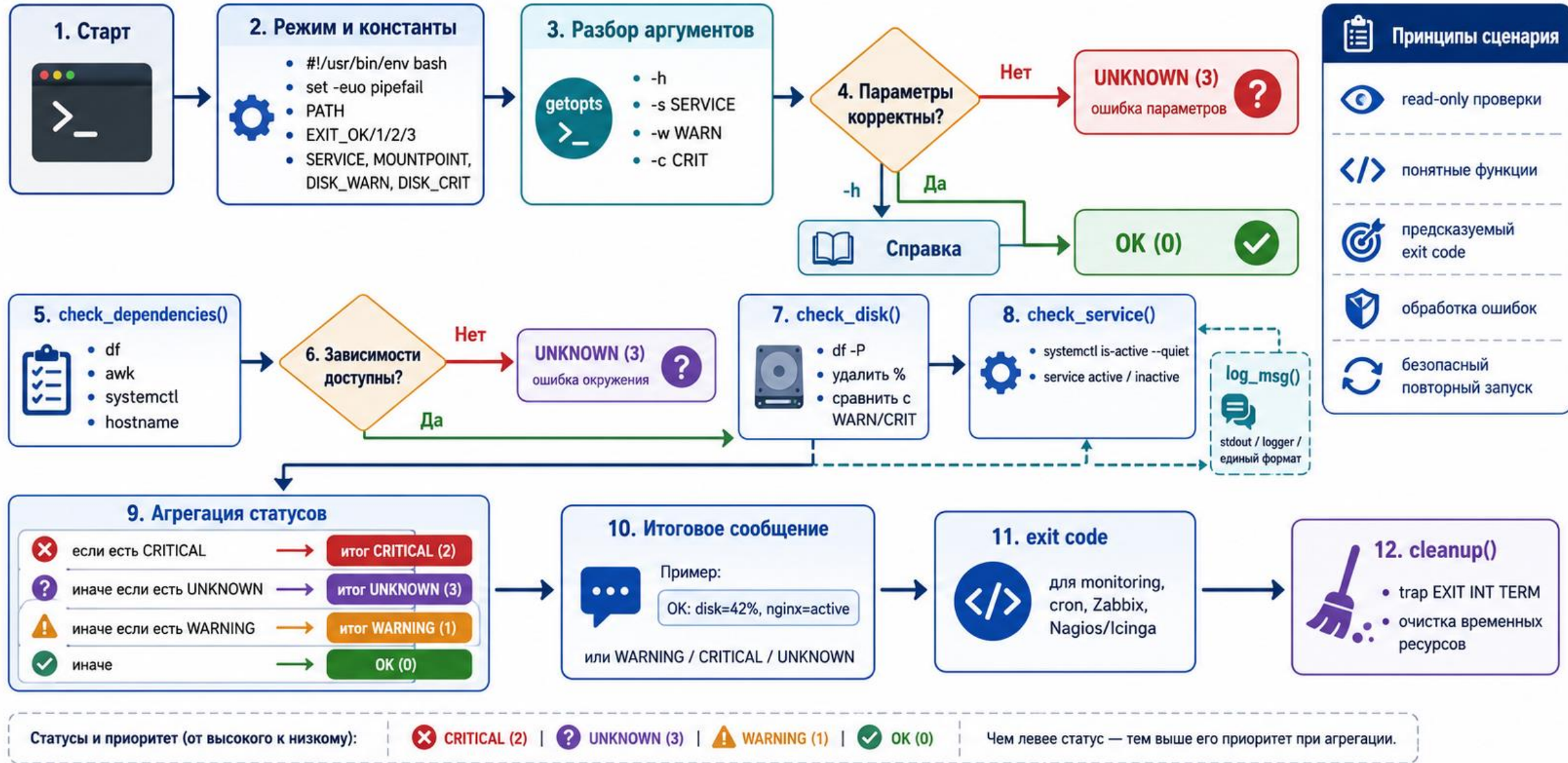
Команда `shellcheck /usr/local/sbin/server-health.sh` обнаруживает незакавыченные переменные и ошибки с \$?

`bash -n` должен быть обязательной проверкой в лаборатории

ShellCheck можно считать рекомендуемой профессиональной проверкой

# Диаграмма: логика server-health.sh

Как сценарий проверяет сервер и возвращает итоговый статус



## **server-health.sh** начинается с режима и констант

```
#!/usr/bin/env bash
set -euo pipefail
PATH='/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/s
bin:/bin'
export PATH
readonly EXIT_OK=0 EXIT_WARNING=1 EXIT_CRITICAL=2
EXIT_UNKNOWN=3
SERVICE='nginx'; MOUNTPOINT='/'; DISK_WARN=80;
DISK_CRIT=90
```



**server-health.sh** проверяет зависимости до основной логики

`check_dependencies()` проверяет наличие `df`, `awk`,  
`systemctl` и `hostname`

Проверка команды: `command -v "$command_name"`  
`>/dev/null 2>&1`

Если команда отсутствует, вывести UNKNOWN в  
STDERR

Функция возвращает EXIT\_UNKNOWN

Зависимости проверяются до чтения метрик

Так ошибка окружения не маскируется как CRITICAL-  
состояние сервера

## **server-health.sh** проверяет диск через **df** и **awk**

`check_disk()` получает `mountpoint`, `warning` и `critical thresholds`

Команда `df -P "$mountpoint"` дает стабильный формат  
`awk 'NR==2 {gsub(/%/,"", $5); print $5}'` возвращает  
число без %

Если `df` или `awk` не сработали, функция возвращает  
**UNKNOWN**

Если `used >= crit`, вернуть **CRITICAL**

Если `used >= warn`, вернуть **WARNING**, иначе **OK**



**server-health.sh** проверяет службу через **systemctl**

`check_service()` получает имя службы как аргумент

Команда `systemctl is-active --quiet "$service"` подходит для проверки активности

Успешная проверка печатает `OK: service nginx is active`

Неактивная служба печатает `CRITICAL: service nginx is inactive`

Функция возвращает 0 или 2

Название `unit` должно соответствовать реальному стенду

**main()** разбирает параметры и валидирует пороги

```
while getopts 'hs:w:c:' opt; do case "$opt" in ... esac;  
done
```

-h печатает usage и завершает сценарий кодом OK

-s задает SERVICE, -w задает DISK\_WARN, -c задает  
DISK\_CRIT

validate\_percent проверяет, что пороги являются  
числами от 1 до 100

Дополнительно проверяется условие  $\text{DISK\_WARN} < \text{DISK\_CRIT}$

Некорректные параметры завершают сценарий кодом  
UNKNOWN

**main()** агрегирует результат двух проверок

```
check_dependencies || exit "$EXIT_UNKNOWN"
```

```
check_disk "$MOUNTPOINT" "$DISK_WARN"
```

```
"$DISK_CRIT" сохраняет disk_rc
```

```
check_service "$SERVICE" сохраняет service_rc
```

Если есть CRITICAL, итоговый exit code равен 2

Если CRITICAL нет, но есть UNKNOWN, итоговый exit code равен 3

Если есть только WARNING, итоговый exit code равен 1;  
иначе 0

## Лабораторная приемка проверяет разные exit codes

Проверить синтаксис: `bash -n server-health.sh`

Проверить обычный запуск: `./server-health.sh; echo $?`

Проверить отсутствующую службу: `./server-health.sh -s nonexistent.service; echo $?`

Проверить ошибочный порог: `./server-health.sh -w abc; echo $?`

Проверить справку: `./server-health.sh -h; echo $?`

Проверить запуск из другого текущего каталога и с минимальным окружением `cron`

## Итоги лекции

Bash-сценарий администратора должен быть читаемым, проверяемым и безопасным при повторном запуске

Quoting, "\$@", local и проверка command substitution уменьшают количество скрытых ошибок

Exit codes 0-3 в лаборатории являются соглашением мониторингового сценария

set -euo pipefail полезен только вместе с явной обработкой ожидаемых ошибок

trap, logger, flock, bash -n и ShellCheck повышают эксплуатационную надежность

## Контрольные вопросы

Почему `set` -е нельзя считать безусловным выходом при любой ошибке?

Чем `"$@"` отличается от `"$*"` при передаче аргументов функции?

Почему результат `df` с процентом нельзя напрямую сравнивать как число?

Чем `return` функции отличается от `exit` сценария?

Почему коды 0-3 являются соглашением мониторингового сценария, а не общим стандартом Bash?

Зачем нужен `flock` при запуске из `cron`?